QALight

# Frontend-розробка

Заняття 25. Об'єкти

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

# Порядок денний



Функції та типи повернення. Вирази типів функцій. Сигнатури викликів та конструкцій. Узагальнені функції. Перевантаження функцій
Об'єкти, масиви та кортежі. Об'єкти та типи властивостей.
Перевантаження функцій.
Створення типу об’єкт.

# Функції

Функції та типи повернення. Вирази типів функцій. Сигнатури викликів та конструкцій. Узагальнені функції. Перевантаження функцій.

# Анотації типів для параметрів функції

Теорія: TypeScript

Функції є основним засобом передачі даних у JavaScript. TypeScript дозволяє вказувати типи як вхідних, так і вихідних значень функцій.
Оголошуючи функцію, ви можете додати **анотації типів** після кожного параметра, щоб визначити, які саме типи даних приймає функція. Анотації типів параметрів розміщуються після назви параметра:

```
// Parameter type annotation
function greet(name: string) {
        console.log("Hello, " + name.toUpperCase() + "!!");
}
```

Якщо параметр має анотацію типу, аргументи, що передаються цій функції, перевірятимуться:

```
// Would be a runtime error if executed!
greet(42);
```

**Аргумент типу «number» не можна призначити параметру типу «string».**
Навіть якщо ви не вказуєте анотації типів для параметрів, TypeScript усе одно перевірятиме, чи передано правильну кількість аргументів.

# Параметри функцій завжди потребують визначення типу

Теорія: TypeScript

Механізм автоматичного визначення типів працює не завжди:

```
const logAlbumInfo = (
 title, // error: Parameter 'title' implicitly has an 'any' type.
 trackCount, // error: Parameter 'trackCount' implicitly has an 'any' type.
 isReleased, // error: Parameter 'isReleased' implicitly has an 'any' type.
) => {
 // rest of function body
};
```

TypeScript видає помилки, якщо для параметрів функції не вказано типи. Це пояснюється тим, що функції суттєво відрізняються від змінних. TypeScript «бачить», яке значення присвоюється тій чи іншій змінній, і може досить точно визначити її тип, однак він не здатен зрозуміти, який тип має бути у параметра функції, спираючись лише на сам параметр.

# Очікувані типи

Теорія: TypeScript

Також система не може визначити тип на основі того, як параметр використовується. Наприклад, якби у вас була функція `add`, що приймає два параметри, TypeScript не зміг би самостійно визначити, що вони мають бути числами:
\

```
function add(a, b) {
 return a + b;
}
```

Аргументи `a` та `b` можуть бути рядками, логічними значеннями (boolean) або чим завгодно іншим. TypeScript не може визначити їхні очікувані типи лише на основі тіла функції.

Тому, хоча механізм виведення типів (inference) працює в більшості випадків, параметри функцій є винятком. У TypeScript їх зазвичай потрібно анотувати (явно вказувати типи).

# Анотації типу значень, що повертається

Теорія: TypeScript

Ви також можете додавати анотації типу значення, що повертається. Вони вказуються після списку параметрів:

```
function getFavoriteNumber(): number {
 return 26;
}
```

Як і у випадку з анотаціями типів для змінних, зазвичай не потрібно вказувати тип значення, що повертається, оскільки TypeScript визначає його автоматично на основі інструкцій `**return**`. Анотація типу в наведеному вище прикладі нічого не змінює. У деяких проєктах тип значення, що повертається, вказують явно - для документації, запобігання випадковим змінам або просто з особистих міркувань.

# Анонімні функції

Теорія: TypeScript

Анонімні функції дещо відрізняються від оголошень функцій. Коли функція з'являється в місці, де TypeScript може визначити, як її буде викликано, параметри функції автоматично отримують типи:

```
const names = ["Alice", "Bob", "Eve"];
// Contextual typing for function - parameter s inferred to have type string
names.forEach(function (s) {
 console.log(s.toUpperCase());
});
// Contextual typing also applies to arrow functions
names.forEach((s) => {
 console.log(s.toUpperCase());
});
```

Хоча параметр `s` не мав анотації типу, TypeScript використав типи функції `forEach` разом із виведеним типом масиву, щоб визначити тип для `s`.

Цей процес називається **контекстним типізуванням**, оскільки саме контекст, у якому використовується функція, визначає, який тип вона повинна мати.

# Функції, що повертають Promise

Теорія: TypeScript

Якщо потрібно вказати тип значення, що повертається функцією, яка повертає Promise, слід використовувати тип `Promise`:

```
async function getFavoriteNumber(): Promise<number> {
 return 26;
}
```

# Перевантаження функцій (overload)

Теорія: TypeScript

Перевантаження функції - це можливість описати кілька сигнатур (варіантів виклику) однієї й тієї самої функції з різними типами аргументів і значень, що повертаються. Простіше кажучи: одна функція може поводитися по-різному залежно від типів аргументів.

TypeScript дозволяє типізувати це через кілька сигнатур "зверху" і одну реалізацію "знизу".

```
// Проста перевантаженість за типом аргументу
// Перевантаження (варіанти виклику)
function reverse(value: string): string;
function reverse(value: number[]): number[];
```

# Реалізація overload

Теорія: TypeScript

```
// Реалізація (одна на всі випадки)
function reverse(value: string | number[]): string | number[] {
 if (typeof value === "string") {
   return value.split("").reverse().join("");
 } else {
   return value.slice().reverse();
 }
}

// Приклади використання
const str = reverse("TypeScript"); // string
const arr = reverse([1, 2, 3]);    // number[]
```

TypeScript розуміє, що:

якщо ти передав string -> функція повертає string
якщо number[] -> повертає number[]

# Сигнатури викликів

Теорія: TypeScript

TypeScript бачить усі сигнатури, але перевіряє їх згори вниз. Коли знаходить підходящу - застосовує її. Реалізація при цьому не видна користувачу - вона слугує лише для виконання.

```
// Різні типи значень, що повертаються
function toArray(value: string): string[];
function toArray(value: number): number[];
function toArray(value: string | number): (string | number)[] {
 return [value];
}

toArray("hi"); // string[]
toArray(42);   // number[]
```

# Сигнатури викликів (різна кількість параметрів)

Теорія: TypeScript

```
// Різна кількість параметрів
function createUser(name: string): { name: string };
function createUser(name: string, age: number): { name: string; age: number };
function createUser(name: string, age?: number) {
 return age ? { name, age } : { name };
}

createUser("Tim");       // { name: string }
createUser("Tim", 25);   // { name: string; age: number }
```

Тут дві сигнатури описують, які комбінації аргументів допустимі, а реалізація об'єднує їх в одному тілі.

# Як писати перевантаження правильно

Теорія: TypeScript

Структура:

```
// 1. Спочатку - сигнатури перевантажень
function fn(param: Type1): ReturnType1;
function fn(param: Type2): ReturnType2;

// 2. Потім - реалізація з union-типами
function fn(param: Type1 | Type2): ReturnType1 | ReturnType2 {
 // логіка
}
```

Важливо: реалізація повинна завжди обробляти всі варіанти. TypeScript перевіряє, що вона сумісна з кожною сигнатурою.

# Помилки при неправильному перевантаженні

Теорія: TypeScript

Не можна писати реалізацію перед сигнатурами перевантаження:

```
function fn(x: any): any { return x; }  // Помилка: реалізація має бути останньою
function fn(x: string): string;
function fn(x: number): number;
// Потрібно навпаки:
function fn(x: string): string;
function fn(x: number): number;
function fn(x: any): any { return x; }
```

Використовуй перевантаження, якщо:

- у функції різні типи аргументів і різні результати;
- не можна виразити це просто через union;
- потрібно зберегти сувору типізацію значення, що повертається.

Об'єкти

Об'єкти, масиви та кортежі. Об'єкти та типи властивостей.

# Типи об'єктів

Теорія: TypeScript

Окрім примітивів, найпоширенішим видом типів є типи об'єктів. Це стосується будь-якого значення JavaScript, що має властивості (а це майже всі значення!). Щоб визначити тип об'єкта, ми просто перелічуємо його властивості та їхні типи.

Наприклад, ось функція, яка приймає об'єкт, що представляє точку:

```
// The parameter's type annotation is an object type
function printCoord(pt: { x: number; y: number }) {
 console.log("The coordinate's x value is " + pt.x);
 console.log("The coordinate's y value is " + pt.y);
}
printCoord({ x: 3, y: 7 });
```

У цьому прикладі ми визначили тип параметра, що має дві властивості - `x` та `y` (обидві - типу `number`). Для розділення властивостей можна використовувати кому (`,`) або крапку з комою (`;`); останній роздільник у будь-якому разі є необов’язковим.

Зазначення типу для кожної властивості також є необов’язковим. Якщо тип не вказано, вважається, що це `any`.

# Необов’язкові властивості

Теорія: TypeScript

У типах об’єктів також можна визначити, що деякі або всі властивості є необов’язковими. Для цього додайте **символ `?`** після назви властивості:

```
function printName(obj: { first: string; last?: string }) {
 // ...
}
// Both OK
printName({ first: "Bob" });
printName({ first: "Alice", last: "Alisson" });
```

# Перевірка даних на `undefined`

Теорія: TypeScript

У JavaScript під час звернення до властивості, якої не існує, ви отримаєте значення `undefined`, а не помилку виконання. Через це, коли ви зчитуєте дані з необов’язкової властивості, вам доведеться перевіряти її на `undefined`, перш ніж використовувати.

```
function printName(obj: { first: string; last?: string }) {
 // Error - might crash if 'obj.last' wasn't provided!
 console.log(obj.last.toUpperCase());
'obj.last' is possibly 'undefined'.
 if (obj.last !== undefined) {
   // OK
   console.log(obj.last.toUpperCase());
 }
 // A safe alternative using modern JavaScript syntax:
 console.log(obj.last?.toUpperCase());
}
```

# Створення типу об'єкта User

Теорія: TypeScript

У TypeScript створити тип об'єкта User та використати його як аргумент функції можна за допомогою ключового слова type або interface.

## Приклад з використанням type

```
// 1. Створення типу об'єкта User
type User = {
 id: number;
 name: string;
 email: string;
 age?: optional | number; // Знак '?' означає необов'язкове поле
};
```

# Використання типу User як аргументу функції

Теорія: TypeScript

```
// 2. Використання типу User як аргументу функції
function printUserInfo(user: User): void {
 console.log(`Користувач: ${user.name}, Email: ${user.email}`);
}
// 3. Приклад виклику функції
const currentUser: User = {
 id: 1,
 name: 'Олена',
 email: 'olena@example.com',
 age: 28,
};

printUserInfo(currentUser
```

# Аліаси типів

Теорія: TypeScript

Аліаси типів дозволяють створити нове ім’я для певного типу. Вони використовуються для надання описової назви складним типам або для створення об’єднань, перетинів та інших складних типів. Аліаси типів можна використовувати з об’єктами, але вони більш універсальні й можуть використовуватися з іншими типами, такими як об’єднання (union), перетини (interception), тапли (typle) та примітивні типи.

```
// Type Alias usage

type Person = {
 name: string;
 age: number;
};

const person: Person = {
 name: "John",
 age: 30,
};
```

# Об’єднаний тип (union type)

Теорія: TypeScript

Система типів TypeScript дозволяє створювати нові типи на основі вже наявних за допомогою різноманітних операторів. Тепер, коли ми вже вміємо визначати окремі типи, настав час навчитися цікавим способам їх поєднання.
Першим способом поєднання типів, з яким ви можете зіткнутися, є об’єднаний тип (union type). Об’єднаний тип — це тип, утворений із двох або більше інших типів; він представляє значення, які можуть належати до будь-якого з цих типів. Кожен із таких типів ми називаємо членом об’єднання. Напишімо функцію, яка може працювати як із рядками, так і з числами:

```
function printId(id: number | string) {
 console.log("Your ID is: " + id);
}

printId(101); // OK
printId("202"); // OK
printId({ myID: 22342 }); // Error
```

Аргумент типу `{ myID: number; }` не можна призначити параметру типу `string | number`.

# Роздільник елементів об'єднаного типу

Теорія: TypeScript

Роздільник елементів об'єднаного типу (union type) дозволено ставити перед першим елементом, тому можна написати й так:

```
function printTextOrNumberOrBool(
 textOrNumberOrBool:
   | string
   | number
   | boolean
) {
 console.log(textOrNumberOrBool);
}
```

# Робота з типами-об’єднаннями (union types)

Теорія: TypeScript

Надати значення, що відповідає типу-об’єднанню, досить просто: достатньо вказати значення типу, який збігається з будь-яким із компонентів цього об’єднання.

TypeScript дозволяє виконувати операцію лише тоді, коли вона є допустимою для кожного з компонентів об’єднання. Наприклад, якщо ви маєте тип-об’єднання `string | number`, ви не зможете використовувати методи, доступні лише для типу `string`:

```
function printId(id: number | string) {
 console.log(id.toUpperCase());
 // Property 'toUpperCase' does not exist on type 'string | number'.
 // Property 'toUpperCase' does not exist on type 'number'.
}
```

Рішення полягає в звуженні типу об’єднання (union type) за допомогою коду - так само, як ви робили б це в JavaScript без анотацій типів. Звуження відбувається тоді, коли TypeScript може визначити більш конкретний тип значення на основі структури коду.

# Звуженні типу об’єднання

Теорія: TypeScript

TypeScript «знає», що лише рядкове значення матиме результат оператора `typeof`, рівний `"string"`:

```
function printId(id: number | string) {
 if (typeof id === "string") {
   // In this branch, id is of type 'string'
   console.log(id.toUpperCase());
 } else {
     console.log(id); // Here, id is of type 'number'
 }
}function welcomePeople(x: string[] | string) {
 if (Array.isArray(x)) {
   // Here: 'x' is 'string[]'
   console.log("Hello, " + x.join(" and "));
 } else {
     console.log("Welcome lone traveler " + x); // Here: 'x' is 'string'
 }
}
```

Зауважте, що у гілці `else` нам не потрібно робити нічого особливого: якщо `x` не був масивом рядків (`string[]`), то він обов’язково був рядком (`string`).

# Об’єднані типи, усі складові яких мають спільну рису

Теорія: TypeScript

Іноді трапляються об’єднані типи (union types), усі складові яких мають спільну рису. Наприклад, і масиви, і рядки мають метод `slice`. Якщо кожен тип в об’єднанні має спільну властивість, ви можете використовувати її без звуження типу:

```
// Return type is inferred as number[] | string
function getFirstThree(x: number[] | string) {
 return x.slice(0, 3);
}
```

Може здатися дивним, що об’єднання типів (union types) фактично успадковує перетин властивостей цих типів. Це не випадковість - назва «об’єднання» (union) походить із теорії типів. Об’єднання типів `number | string` утворюється шляхом об’єднання множин значень, притаманних кожному з цих типів. Зауважте: якщо ми маємо дві множини з певними характеристиками, то до об’єднання цих множин застосовуються лише ті характеристики, що є спільними для обох (тобто їхній перетин).

# Об'єднання типів

Теорія: TypeScript

```
// Union of Type
type Status = "pending" | "in_progress" | "completed";
```

Якщо потрібно створити нову назву для певного типу або працювати з об’єднаннями (union) та перетинаннями (intersection), аліаси типів забезпечують більшу гнучкість.

# Тип Arrays

Теорія: TypeScript

Робота з масивами в TypeScript схожа на JavaScript, але завдяки типізації ви можете чітко вказати, які саме дані мають зберігатися в масиві. Це захищає від випадкового додавання неправильних значень (наприклад, тексту в масив чисел).

Щоб вказати тип масиву, наприклад `[1, 2, 3]`, можна скористатися синтаксисом `number[]`; цей запис працює для будь-якого типу (наприклад, `string[]` - це масив рядків тощо). Також можна зустріти запис `Array<number>`, який означає те саме.

Зауважте, що `[number]` - це зовсім інше.

# Способи оголошення масивів

Теорія: TypeScript

Існує два рівнозначних синтаксиси для створення масивів. Найчастіше використовують перший:

* Квадратні дужки (тип[]):

```
let numbers: number[] = [1, 2, 3];
let strings: string[] = ["Київ", "Львів", "Одеса"];
```

* Дженерик-запис (Array<тип>):

```
let scores: Array<number> = [85, 90, 78];
```

# Захист від помилок типів

Теорія: TypeScript

Якщо ви спробуєте додати в масив елемент іншого типу, TypeScript одразу видасть помилку:

```
let fruits: string[] = ["яблуко", "банан"];

// ❌ Помилка: Аргумент типу 'number' не можна передати в параметр типу 'string'
fruits.push(42);
// ❌ Помилка: Тип 'boolean' не можна призначити типу 'string'
fruits[0] = true;
```

# Масиви складних об'єктів

Теорія: TypeScript

Ви можете комбінувати масиви з інтерфейсами (або типами), щоб створювати масиви об'єктів:

```
type Book = {
 title: string;
 year: number;
}
// Створюємо масив, де кожен елемент — це об'єкт Book
const library: Book[] = [
 { title: "Кобзар", year: 1840 },
 { title: "Тигролови", year: 1944 }
];
```

# Масиви з кількома типами даних (Union Types)

Теорія: TypeScript

Якщо в масиві дійсно мають зберігатися різні типи даних (наприклад, і числа, і рядки), використовують вертикальну риску |:

```
// Масив може містити або числа, або рядки
let mixed: (number | string)[] = [1, "два", 3, "чотири"];
```

# Кортежі (Tuples) - масиви фіксованої довжини

Теорія: TypeScript

Якщо вам потрібен масив із чітко визначеною кількістю елементів та конкретним порядком їхніх типів, використовуйте кортежі:

```
// Перший елемент обов'язково має бути рядком, другий — числом
let userSession: [string, number];

userSession = ["admin", 12345]; // ✅ Правильно
// ❌ Помилка: Порядок типів порушено
userSession = [12345, "admin"];
```

# Автоматичне виведення типів (Type Inference)

Теорія: TypeScript

Якщо ви одразу створюєте масив із даними, TypeScript самостійно зрозуміє його тип, і писати його вручную не обов'язково:

```
// TypeScript сам зрозуміє, що це boolean[]
const LightSwitches = [true, false, true];
```

Практика

Перевантаження функцій.
Створення типу об’єкт.

# Структура перевантаження функцій

Практика: TypeScript

У TypeScript перевантаження функцій дозволяє описати кілька способів виклику функції з різними типами або кількістю аргументів, забезпечуючи при цьому сувору типізацію.

Конструкція складається з двох частин:

1. Сигнатури перевантаження (Overload Signatures): Тільки описують типи, не мають тіла функції і не компілюються в JavaScript.

2. Сигнатура реалізації (Implementation Signature): Містить безпосередньо код функції. Її типи повинні бути достатньо загальними, щоб покрити всі варіанти з перевантажень.

# Створення об'єкта User з різною кількістю параметрів

Практика: TypeScript

Для вирішення цієї задачі найкраще підходить комбінація **перевантаження функцій та необов'язкових параметрів** (optional parameters).
Ось рішення, де користувача можна створити трьома різними способами: лише за id, за id та email, або передавши повний об'єкт профілю.

```
// Описуємо тип користувача
interface User {
 id: string;
 email?: string;
 name?: string;
 age?: number;
}
```

# Сигнатура перевантаження

Практика: TypeScript

```
// 1. Сигнатури перевантаження (описують допустимі варіанти виклику)
function createUser(id: string): User;
function createUser(id: string, email: string): User;
function createUser(id: string, profile: { name: string; age: number }): User;
```

# Сигнатура реалізації

Практика: TypeScript

```
function createUser(id: string, param2?: string | { name: string; age: number }): User {
 // Базовий об'єкт, який буде створено в будь-якому випадку
 const user: User = { id };

 if (typeof param2 === 'string') {
   // Якщо другий параметр — рядок, то це email
   user.email = param2;
 } else if (typeof param2 === 'object' && param2 !== null) {
   // Якщо другий параметр — об'єкт, копіюємо дані профілю
   user.name = param2.name;
   user.age = param2.age;
 }

 return user;
}
```

Саму сигнатуру реалізації викликати напряму не можна, якщо вона не збігається з одним із перевантажень.

# Тестування викликів

Практика: TypeScript

```
// === Тестування викликів ===
// 1. Створення лише за ID
const user1 = createUser("usr_001");
// Результат: { id: "usr_001" }
// 2. Створення за ID та Email
const user2 = createUser("usr_002", "alex@example.com");
// Результат: { id: "usr_002", email: "alex@example.com" }
// 3. Створення за ID та повним профілем
const user3 = createUser("usr_003", { name: "Олена", age: 28 });
// Результат: { id: "usr_003", name: "Олена", age: 28 }
// ❌ Помилка компіляції (TypeScript заблокує цей виклик)
// const user4 = createUser("usr_004", 42);
```

# Тестування викликів

Практика: TypeScript

## 🔑 Чому це працює ефективно?

1. Другий параметр param2? позначено як необов'язковий за допомогою знака ?, що дозволяє викликати функцію лише з одним аргументом (user1).
2. Тип другого параметра в реалізації є Union типом (string | { name: string; age: number }). Це дозволяє TypeScript приймати або рядок, або об'єкт, але чітко контролювати, щоб туди не передали щось третє (наприклад, number).
3. TypeScript автоматично підказуватиме правильні поля (автодоповнення) залежно від того, який саме варіант виклику ви почнете набирати в IDE.

# Сумісність типів

Практика: TypeScript

Типи в реалізації повинні бути ширшими (наприклад, через Union типи string | number), щоб коректно обробляти кожен варіант.

# Порядок має значення

Практика: TypeScript

TypeScript перевіряє сигнатури зверху вниз до першого збігу. Тому специфічніші варіанти слід ставити вище, а загальні — нижче.

# Ролі користувачів

Практика: TypeScript

Розширимо функцію новими сигнатурами.
Коли комбінацій параметрів стає багато (особливо якщо вони мають однакові типи, як-от телефон та email), найкраща практика в TypeScript — використовувати об'єкти конфігурації (**options objects**) для складних випадків.
Це дозволяє уникнути плутанини з порядком аргументів.
Додамо роль (Role), телефон (phone) та дату реєстрації (createdAt).

```
// Визначаємо ролі користувачів
type UserRole = 'admin' | 'editor' | 'user';
// Головний інтерфейс об'єкта User
interface User {
 id: string;
 email?: string;
 phone?: string;
 role: UserRole;
 createdAt: Date;
 name?: string;
 age?: number;
}
```

# Інтерфейс для детальних налаштувань

Практика: TypeScript

```
// Інтерфейс для детальних налаштувань (додаткова комбінація даних)
interface DetailedUserInfo {
 email?: string;
 phone?: string;
 role?: UserRole;
 name?: string;
 age?: number;
 createdAt?: Date;
}
```

# Сигнатури перевантаження

Практика: TypeScript

```
// Варіант A: Мінімальний — тільки ID (роль та дата створюються автоматично)
function createUser(id: string): User;
// Варіант B: ID + Email + Роль (часта комбінація для швидкої реєстрації)
function createUser(id: string, email: string, role: UserRole): User;
// Варіант C: ID + Специфічний об'єкт з усіма можливими деталями
function createUser(id: string, details: DetailedUserInfo): User;
```

# Сигнатура реалізації

Практика: TypeScript

```
function createUser(
 id: string,
 param2?: string | DetailedUserInfo,
 param3?: UserRole
): User {
 // Задаємо значення за замовчуванням для обов'язкових полів
 const defaultUser: User = {
   id,
   role: 'user',
   createdAt: new Date()
 };
```

# Сигнатура реалізації

Практика: TypeScript

```
// Перевіряємо Варіант B: другий параметр — рядок (email), третій — роль
if (typeof param2 === 'string') {
  return {
    ...defaultUser,
    email: param2,
    role: param3 || 'user'
  };
}
```

# Сигнатура реалізації

Практика: TypeScript

```
 // Перевіряємо Варіант С: другий параметр — об'єкт з деталями
 if (typeof param2 === 'object' && param2 !== null) {
   return {
     ...defaultUser,
     ...param2, // Перезаписуємо дефолтні поля тими, що прийшли в об'єкті
     role: param2.role || defaultUser.role,
     createdAt: param2.createdAt || defaultUser.createdAt
   };
 }

 // Перевіряємо Варіант А: просто повертаємо дефолтного юзера
 return defaultUser;
}
```

# Тестування реалізації перевантаження

Практика: TypeScript

```
// === Тестування  комбінацій ===
// ✅ Варіант A: Тільки ID
const quickUser = createUser("u_001");
// { id: "u_001", role: "user", createdAt: 2026-10-12... }
// ✅ Варіант B: ID, Email та специфічна Роль
const adminUser = createUser("u_002", "admin@app.com", "admin");
// { id: "u_002", email: "admin@app.com", role: "admin", createdAt: 2026-10-12... }
// ✅ Варіант C: ID + Об'єкт з телефоном та датою реєстрації
const customUser = createUser("u_003", {
 phone: "+380501234567",
 createdAt: new Date("2025-01-01"),
 name: "Дмитро"
});
// { id: "u_003", phone: "+380501234567", name: "Дмитро", role: "user", createdAt: 2025-01-01... }
```

# Тестування реалізації перевантаження

Практика: TypeScript

1. Гнучкість об'єкта деталей: Завдяки DetailedUserInfo у варіанті C ви можете комбінувати параметри як завгодно - передати лише phone, або phone + createdAt, або взагалі всі поля разом. Розробнику не потрібно згадувати, на якому місці (другому чи п'ятому) стоїть телефон.
2. Безпека типів: TypeScript не дозволить у варіанті B передати випадковий рядок замість ролі (компілятор очікує суворо 'admin' | 'editor' | 'user').
3. Значення за замовчуванням: Оскільки role та createdAt є обов'язковими в інтерфейсі User, функція сама підставляє поточну дату та роль 'user', якщо розробник їх не вказав.

# ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

# Посилання на довідники



Typescriptlang: https://www.typescriptlang.org/

Tsconfig: https://www.typescriptlang.org/tsconfig/

Microsoft for JavaScript developers: https://developer.microsoft.com/en-us/javascript/

JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Nodejs: ](https://nodejs.org/uk)

# Дякую за увагу!

# До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua